

UNIT2



**Ganpat
University**

॥ विद्यया समाजोत्कर्षः ॥

Department of
Computer Science



B.Sc. Hons. (CA&IT) Sem-I
U11A1IP1-INTRODUCTION TO
PROGRAMMING-I

UNIT-2

Overview of C

Prepared By:

Akash Chaudhary

**Department of Computer Science Ganpat
University**

❖ Introduction to C Programming

➤ What is C ?

- C is a general purpose, Procedure-oriented programming language.
- C programming language was developed in **1972** by **Dennis Ritchie** at **bell laboratories** of AT&T (American Telephone & Telegraph), located in the U.S.A. It was initially designed for system programming to write the **UNIX operating system**.

- C programming is considered as the base for other programming languages, that is why it is known as the **mother language**.

- It can be defined by the following ways:

1. Mother language
2. System programming language
3. Procedure-oriented programming language
4. Structured programming language
5. Mid-level programming language

➤ Features / Importance of C

- Simple
- Machine Independent or Portable
- Mid-level programming language
- Structured programming language
- Rich Library
- Quicker Compilation

1) Simple

C is a simple language because it provides a structured approach (to break the problem into parts), a rich set of library functions, data types, etc.

2) Portable

C is a portable language and programs made in it can be run on different machines.

3) Mid-level / Intermediate programming language

C includes both low-level programming and high-level language so it is known as a mid-level and intermediate programming language. It is used to develop system applications such as kernel, driver, etc.

4) Structured programming language

C is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify. Functions also provide code reusability.

5) Rich Library

C provides a lot of inbuilt functions that make the development fast.

6) Quicker Compilation

C programs tend to be compact and run quickly. Hence the compilation and execution time of the C language is fast.

➤ Compilation process

The compilation is a process of **converting the source code into object code**. It is done with the help of the compiler.

The compiler checks the source code for the syntactical or structural errors, and if the source code is error-free, then it generates the object code.



Download Turbo C++ software

- Link : [Download Turbo C++](#)

➤ First C Program

```
#include <stdio.h>           // Standard Input/Output library (for printf)
#include <conio.h>           // Console Input/Output library (for clrscr & getch)
void main()                 // Program starts here
{
    clrscr();               // Clears the screen (like wiping a whiteboard)
    printf("Hello World!"); // Prints text on screen
    getch();               // Waits for user to press any key (like a pause button)
}
```

Output:-

Hello World!

#include <stdio.h> includes the standard input output library functions.
The printf() & scanf() function is defined in stdio.h

#include <conio.h> includes the console input output library functions.
The getch() & clrscr() function is defined in conio.h

void main() The main() function is the entry point of every program in c language.

➤ Structure

- **Documentation Section** – Comments about the program.
- **Preprocessor Directives** – e.g., #include<stdio.h> for I/O functions.
- **Global Declarations** – Variables and functions declared before main().
- **main() Function** – The starting point of every program.
- **Local Declarations** – Variables declared inside functions.
- **Statements & Expressions** – Actual instructions to perform tasks.
- **Functions** – User-defined reusable modules.

➤ How to compile and run the c program

- 1) Open Turbo C++: Launch the Turbo C++ application.
- 2) Write your code: Type your C program into the editor window.
- 3) Save the file: Save the file with a **.c extension** (for C).
- 4) Compile: Press **Alt + F9** to compile the code. If there are errors, they will be displayed in the message window. Correct the errors in your code and recompile.
- 5) Run: Once the code compiles without errors, press **Ctrl + F9** to run the program.
- 6) View output: After the program runs, press Alt + F5 to see the output in the user screen.

➤ Components of a C Program (Detailed)

- Preprocessor Directives: #include, #define.
- Variables: Store data (e.g., int x = 5;).
- Functions: Reusable blocks (main(), printf()).
- Comments: // Single-line or /* Multi-line */.
- Statements & Expressions: Instructions (x = y + 10;).

➤ Terminology

1. Source Program

- This is the original program written by a programmer using a high-level programming language like C, Java, or Python.

2. Compiler

- A tool that translates the source program into machine code that the computer can understand, but doesn't run it yet.

3. Object Program

- The machine code version of the source program, created by the compiler. It is not fully ready to run because it might need to be linked with other code.

4. Executable Program

- A complete program that can be directly run by the computer. It is created after the object program is linked with all necessary resources.

5. Linker

- A tool that combines one or more object programs and necessary libraries to create an executable program.

6. Loader

- A part of the operating system that loads the executable program into memory and starts it.

7. Debugger

- A tool that helps find and fix errors (bugs) in a program by allowing the programmer to examine how the program runs step-by-step.

8. Interpreter

- A program that reads and runs the source program line by line, without converting the entire program into machine code first.

➤ **Variables in C**

- A variable is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times.
- It is a way to represent memory location through symbol so that it can be easily identified.

Syntax:	Data_type variable_list;
Example:	Int a;

Rules for defining variables

- A variable can have alphabets, digits, and underscore.
- A variable name can start with the alphabet, and underscore only. It can't start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Example of variable names:

Valid Names	Invalid Names
Int a; Int b1; Int _a1;	Int a b; Int 5; Int long;

➤ Data Types

- Tell the compiler what kind of data a variable can store.

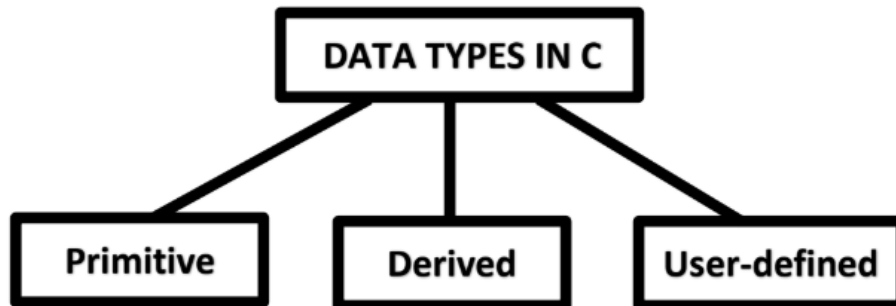
Basic data types in C:	
• int – for integers • float – for decimal numbers • char – for characters • double – for large decimals	<pre>int = 20; float = 5.8; char = 'A'; double = 25000.50;</pre>

Data Type	Format Specifier	Example in printf()	Example Output
int	%d or %i	printf("%d", num);	10
long int	%ld	printf("%ld", num);	100000
char	%c	printf("%c", letter);	A
float	%f	printf("%.2f", height);	5.80
double	%lf	printf("%.4lf", salary);	12345.6789
long double	%Lf	printf("%.5Lf", amount);	12345.67890
string (char[])	%s	printf("%s", name);	shreya

- Table of the data types, their memory sizes, and their value ranges.

Data Type	Memory Size	Range
char	1 byte	-128 to 127
int	2 bytes	-32,768 to 32,767
short int	2 bytes	-32,768 to 32,767
long int	4 bytes	-2,147,483,648 to 2,147,483,647

float	4 bytes	3.4e-38 to 3.4e+38 (approx.)
double	8 bytes	1.7e-308 to 1.7e+308 (approx.)
long double	10 bytes	3.4e-4932 to 1.1e+4932 (approx.)



Primitive Data Type	int, char, float, void
Derived Data Type	array, pointers, string
User-defined data Type	structure, union, enum, typedef, function

➤ User Defined Type declaration (typedef).

- The `typedef` keyword is used to create a new name (alias) for an existing data type.
- Purpose: Improves program readability and makes code more understandable.

Syntax:

```
typedef existing_type new_name;
```

Examples:

```
typedef int uint;
typedef char uchar;
uint age = 25;
uchar grade = 'A';
```

Advantages of typedef:

1. Makes code shorter and easier to read.
2. Increases portability (same alias can map to different types in different systems).
3. Useful in defining complex data structures like struct, enum, and pointers.

C Tokens

The smallest units in a C program.

- Keywords
- Identifiers
- Constants
- Operators
- Special Symbols

➤ Keywords

● A keyword is a reserved word. You cannot use it as a variable name, constant name, etc. There are only 32 reserved words (keywords) in the C language.

- A list of 32 keywords in the c language is given below:

auto	break	case	char	const	continue	default	do	auto	break
double	else	enum	extern	float	for	goto	if	double	else
int	long	register	return	short	signed	sizeof	static	int	long
struct	switch	typedef	union	unsigned	void	volatile	while	struct	switch

➤ Identifiers

Identifiers are names given to program elements like:

- Variables (e.g., count, total_amount)
- Functions (e.g., calculateSum())
- Arrays, structures, unions, labels, etc.
- Rules for Valid C Identifiers

	Valid	Invalid
First Character: Must be a letter (A-Z, a-z) or underscore _	sum, _count	2sum (starts with digit)
Subsequent Characters: Can be letters, digits (0-9), or underscores	total1, student_name	first-name (hyphen not allowed)
No Spaces/Special Symbols:	average_score	average score (space).

- Case-Sensitive: - Age $\neq$ age $\neq$ AGE
- Length Limit: Up to 31 characters (compiler-dependent)

Keyword	Identifier
Keyword is a pre-defined word.	The identifier is a user-defined word
It must be written in a lowercase letter.	It can be written in both lowercase and uppercase letters.
Its meaning is pre-defined in the c compiler.	Its meaning is not defined in the c compiler.
It is a combination of alphabetical characters.	It is a combination of alphanumeric characters.
It does not contain the underscore character.	It can contain the underscore character.

➤ Constants

1. Constants using `const` keyword

- If you don't want others (or yourself) to change existing variable values, you can use the `const` keyword.
- This will declare the variable as "constant", which means unchangeable and read-only:
 - `const float PI = 3.14159;`

2. Constants using `#define`

- These are defined before compilation and replaced throughout the code
- `#define PI 3.14159`

➤ C Operators

• An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.

- There are different types of operations in C language.

1. Arithmetic Operators (+ , - , * , / , %)
2. Relational Operators (> , < , >= , <= , == , !=)
3. Logical Operators (&& , || , !)
4. Bitwise Operators (& , | , >> , << , ~)
5. Ternary or Conditional Operators (? , :)
6. Assignment Operator (= , += , -= , *= , /= , %=)
7. Increment/Decrement Operator (++ , --)

➤ Special Symbols in C

The table given below is the respective meaning of the special symbols used in the C programming language.

Symbol	Meaning
~	Tilde
!#\$	Exclamation mark, Number sign, Dollar sign
%^&	Percent sign, Caret, Ampersand
*()	Asterisk, Left parenthesis, Right parenthesis
_+,	Underscore, Plus sign, Comma
./	Period, Slash, Vertical bar
\`-	Backslash, Apostrophe, Minus sign
=<>	Equal to sign, Opening angle bracket, Closing angle bracket
?{}	Question mark, Left brace, Right brace

[]:	Left bracket,Right bracket,Colon
";	Quotation mark,Semicolon

➤ **printf() and scanf() in C**

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

➔ **printf() Function**

- The printf() function is used for output.
- It prints the given statement to the console.

The syntax of printf() function:-

printf("Format Specifier",variable_name);

[The format specifier can be %d (integer), %c (character), %s (string), %f (float) etc.]

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a=50;
    clrscr();
    printf("value of a=%d",a);
    getch();
}
```

Output:-

Value of a=50

➔ **scanf() function**

- The scanf() function is used for input. It reads the input data from the console.

The syntax of scanf() function:-

scanf("Format Specifier",&variable_name);

```
#include<stdio.h>
#include<conio.h>
void main()
{
    Int a;
    clrscr();
    printf("Enter the a value:")
    scanf("%d",&a);
    printf("Value of a=%d",a);
    getch();
}
```

Output:-

Enter the a value:21
Value of a=21